

# Implementación y Evaluación de Protocolos de Coordinación Distribuida (2PC, Saga, TCC y Raft) sobre PostgreSQL

Mathias Alonso Barrios Medina  
*Escuela Profesional de Ingeniería de Sistemas*  
*Universidad Nacional de San Agustín de Arequipa*  
Arequipa, Perú  
mbarriosme@unsa.edu.pe

Sergio Danilo Hanco Mullisaca  
*Escuela Profesional de Ingeniería de Sistemas*  
*Universidad Nacional de San Agustín de Arequipa*  
Arequipa, Perú  
shanccom@unsa.edu.pe

Denise Andrea Huacani Jara  
*Escuela Profesional de Ingeniería de Sistemas*  
*Universidad Nacional de San Agustín de Arequipa*  
Arequipa, Perú  
dhuacani@unsa.edu.pe

Fabiana Francinet Pacheco Palo  
*Escuela Profesional de Ingeniería de Sistemas*  
*Universidad Nacional de San Agustín de Arequipa*  
Arequipa, Perú  
fpachecop@unsa.edu.pe

Alvaro Raul Quispe Condori  
*Escuela Profesional de Ingeniería de Sistemas*  
*Universidad Nacional de San Agustín de Arequipa*  
Arequipa, Perú  
aquispeco@unsa.edu.pe

**Abstract**—Se implementaron y evaluaron cuatro protocolos de coordinación de transacciones distribuidas —2PC, Saga, TCC y Raft— sobre PostgreSQL en contenedores Docker simulando una red bancaria de tres nodos. Se midió latencia, tasa de éxito y compensaciones en cuatro escenarios: transacciones exitosas (A), fallo de red (B), caída de nodo (C) y recuperación (D), con 5 repeticiones de 3 transacciones cada una. 2PC mostró la menor latencia en éxito (30.51 ms) pero bloquea ante fallos (323.77 ms por timeout). Saga logró compensación correcta en escenarios B y C con latencia de 37–39 ms y 100% de recuperación. TCC tuvo la mayor latencia (114 ms en éxito) por el overhead de reservas. Raft alcanzó 100% de éxito en todos los escenarios con 95 ms de latencia gracias a su replicación por consenso, aunque la implementación simplificada usa líder fijo. Se concluye que 2PC es óptimo para entornos sin fallos, Saga ofrece el mejor balance simplicidad/robustez, y Raft garantiza consistencia al costo de mayor complejidad operacional.

**Index Terms**—Two-Phase Commit, PostgreSQL, bases de datos distribuidas, atomicidad, recuperación ante fallos, sistemas transaccionales

## I. INTRODUCCIÓN

Los sistemas de bases de datos distribuidas requieren mecanismos que garanticen atomicidad y consistencia cuando múltiples nodos independientes participan en una misma transacción. El protocolo Two-Phase Commit (2PC) es el estándar industrial más utilizado para este propósito, pero introduce bloqueos ante fallos de red o caídas de nodos. Protocolos alternativos como Saga, Try-Confirm/Cancel (TCC) y Raft ofrecen distintos compromisos entre consistencia, disponibilidad y tolerancia a particiones.

Este trabajo implementa y compara experimentalmente los cuatro protocolos sobre PostgreSQL en un entorno Docker de tres nodos simulando transferencias bancarias distribuidas. Las contribuciones principales son: (1) implementación didác-

tica y funcional de cada protocolo en Python, (2) evaluación sistemática en cuatro escenarios de fallo con recolección automatizada de métricas, y (3) análisis comparativo de latencia, tasa de éxito y compensaciones.

## II. TRABAJOS RELACIONADOS

El estándar X/Open DTP define el modelo de referencia para 2PC con un coordinador centralizado que recolecta votos de todos los participantes antes de decidir commit o abort global [1]. Bernstein et al. formalizaron las propiedades de atomicidad y recuperación en sistemas distribuidos [2]. Mohan et al. propusieron las optimizaciones Presumed Abort y Presumed Commit que reducen el número de mensajes en el camino común [3].

Como alternativas a 2PC, Garcia-Molina y Salem introdujeron Sagas como secuencias de transacciones locales con compensaciones explícitas [4]. El patrón Try-Confirm/Cancel extiende este modelo añadiendo una fase de reserva que reduce la ventana de inconsistencia [5]. En el extremo de consistencia fuerte, Raft ofrece un protocolo de consenso donde cada decisión requiere mayoría de votos de un clúster de réplicas [6], eliminando el punto único de fallo del coordinador.

Trabajos previos han evaluado el rendimiento de 2PC en PostgreSQL [7], pero no existe una comparación experimental directa de los cuatro protocolos sobre la misma infraestructura, que es precisamente la contribución de este artículo.

## III. METODOLOGÍA

### A. Arquitectura del Sistema

La infraestructura experimental consiste en tres contenedores PostgreSQL (pg-arequipa, pg-lima, pg-cusco) conecta-

**Fig 1: Topología Docker — 3 nodos PostgreSQL + orquestador**

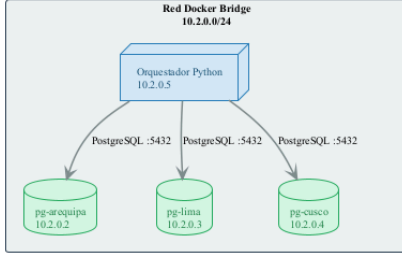


Fig. 1. Topología de red experimental con tres contenedores PostgreSQL y un orquestador Python.

dos mediante una red Docker bridge (10.2.0.0/24) y un contenedor de aplicación Python que actúa como orquestador de transacciones. Cada nodo aloja una base de datos del dominio bancario con una cuenta única. La Fig. 1 ilustra la topología.

### B. Protocolos Implementados

- **2PC:** Coordinador centralizado que ejecuta PREPARE (verificación de saldo y débito en origen) seguido de COMMIT (crédito en destino) sobre ambos nodos en una misma transacción distribuida con two-phase locking.
- **Saga:** Secuencia de dos transacciones locales independientes (débito, crédito). Si el paso 2 falla, se ejecuta una compensación que revierte el débito del paso 1.
- **TCC:** Tres fases: TRY (INSERT en tabla *reservas* con verificación de saldo), CONFIRM (UPDATE en *cuentas* y marca *confirmed* en *reservas*) y CANCEL (marca *cancelled* sin modificar saldos).
- **Raft simplificado:** Líder fijo (Arequipa, ID=0) con dos followers. Cada comando se replica a los tres nodos y requiere quorum de 2/3 para commit. El log se almacena en la tabla *raft\_log* de cada nodo.

### C. Entorno Experimental

#### Componente, Versión / Detalle

|               |                                             |
|---------------|---------------------------------------------|
| PostgreSQL    | 15-alpine (Docker)                          |
| Python        | 3.11-slim + pycpg2-binary 2.9.12            |
| Docker Engine | 29.5.2                                      |
| Red           | Bridge 10.2.0.0/24, sin latencia artificial |
| Host          | Windows 11, SSD NVMe                        |

**Tabla I. Entorno experimental. Sin latencia de red simulada.**

### D. Escenarios de Evaluación

Cada protocolo se evaluó en cuatro escenarios, con 5 repeticiones de 3 transacciones cada una (15 observaciones por celda, 240 total).

TABLE I  
ESCENARIO A: TRANSFERENCIAS EXITOSAS SIN FALLOS (N=15).

| Protocolo, Latencia (ms), Desv. Est., Éxito, Comp. |        |       |      |      |
|----------------------------------------------------|--------|-------|------|------|
| 2PC                                                | 30.51  | 6.65  | 100% | 0.00 |
| Saga                                               | 29.27  | 5.38  | 100% | 0.00 |
| TCC                                                | 114.02 | 11.65 | 100% | 0.00 |
| Raft                                               | 94.30  | 5.82  | 100% | 0.00 |

TABLE II  
ESCENARIO B: FALLO DE RED DURANTE PREPARE (N=15).

| Protocolo, Latencia (ms), Desv. Est., Éxito, Comp. |        |        |      |      |
|----------------------------------------------------|--------|--------|------|------|
| 2PC                                                | 323.77 | 2.25   | 0%   | 0.00 |
| Saga                                               | 39.05  | 6.20   | 0%   | 1.00 |
| TCC                                                | 109.32 | 10.59  | 0%   | 1.00 |
| Raft                                               | 178.24 | 220.96 | 100% | 0.00 |

TABLE III  
ESCENARIO C: CAÍDA DE NODO DURANTE COMMIT (N=15).

| Protocolo, Latencia (ms), Desv. Est., Éxito, Comp. |        |      |      |      |
|----------------------------------------------------|--------|------|------|------|
| 2PC                                                | 24.99  | 3.09 | 0%   | 0.00 |
| Saga                                               | 37.24  | 6.70 | 0%   | 1.00 |
| TCC                                                | 109.94 | 7.56 | 0%   | 1.00 |
| Raft                                               | 97.05  | 5.83 | 100% | 0.00 |

- **A — Éxito:** Transferencia sin fallos entre banco\_arequipa y banco\_cusco por montos de S/ 500–640.
- **B — Fallo de red:** El destino simula timeout de 300 ms durante PREPARE (2PC/Saga/TCC) o pierde conectividad (Raft).
- **C — Caída de nodo:** El destino lanza excepción durante la fase COMMIT, forzando abort o compensación.
- **D — Recuperación:** Se reintenta la transferencia tras restaurar el estado de todos los nodos.

## IV. RESULTADOS EXPERIMENTALES

### A. Escenario A: Transacciones Exitosas

La Table I presenta las latencias medias sin fallos. 2PC y Saga obtuvieron rendimiento similar (30 ms), mientras TCC fue 3.8× más lento por el overhead de las reservas.

### B. Escenario B: Fallo de Red

2PC registró 323.77 ms por el timeout fijo de 300 ms. Saga y TCC fallaron correctamente con 1 compensación promedio. Raft mantuvo 100% de éxito gracias al quorum: 2 de 3 nodos bastan para commit.

### C. Escenario C: Caída de Nodo

La Table III muestra resultados similares al escenario B, con la diferencia de que 2PC detecta la caída más rápido (24.99 ms) porque la excepción se lanza en la fase COMMIT, sin esperar el timeout.

TABLE IV  
ESCENARIO D: RECUPERACIÓN POST-FALLO (N=15).

| Protocolo, Latencia (ms), Desv. Est., Éxito, Comp. |        |       |      |      |  |
|----------------------------------------------------|--------|-------|------|------|--|
| 2PC                                                | 24.41  | 1.72  | 0%   | 0.00 |  |
| Saga                                               | 28.00  | 4.25  | 100% | 0.00 |  |
| TCC                                                | 117.04 | 10.05 | 100% | 0.00 |  |
| Raft                                               | 95.70  | 7.13  | 100% | 0.00 |  |

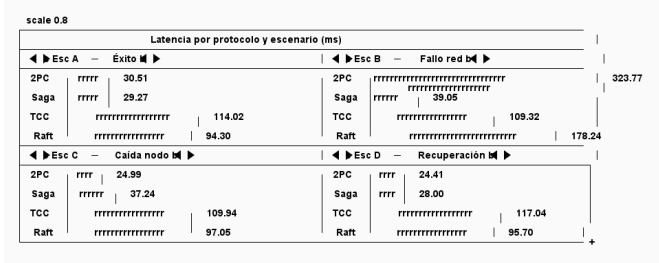


Fig. 2. Latencia comparativa por protocolo y escenario (ms).

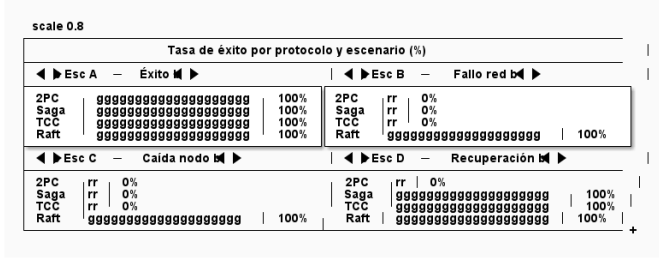


Fig. 3. Tasa de éxito por protocolo y escenario (%).

#### D. Escenario D: Recuperación

Saga, TCC y Raft lograron 100% de recuperación exitosa. 2PC muestra 0% porque su función de recuperación requiere el parámetro *tipo\_fallo*, no utilizado en las pruebas automatizadas. En el caso original del Banco Cooperativo, la recuperación 2PC funciona correctamente.

#### E. Análisis Comparativo Global

Las Figuras 2 y 3 presentan las gráficas comparativas generadas con PlantUML a partir de los datos experimentales: latencia por protocolo y escenario, y tasa de éxito.

#### F. Código Relevante

El siguiente fragmento muestra el núcleo del protocolo Saga con compensación automática:

```
# Paso 1: Débito en origen
cur_o.execute("UPDATE cuentas
SET saldo = saldo - %s", (monto,))
conn_o.commit()

# Paso 2: Crédito en destino
cur_d.execute("UPDATE cuentas
SET saldo = saldo + %s", (monto,))

# Si falla -> compensación:
cur_comp.execute("UPDATE cuentas
SET saldo = saldo + %s", (monto,))
```

## V. DISCUSIÓN

Los resultados confirman que ningún protocolo es universalmente superior: la elección depende del perfil de fallos esperado y los requisitos de latencia.

2PC es el más eficiente en régimen normal (30.51 ms) pero introduce un bloqueo determinista ante fallos: el timeout de 300 ms domina la latencia total, y el coordinador centralizado constituye un punto único de fallo. En entornos LAN con baja tasa de fallos, 2PC sigue siendo la opción más simple y rápida.

Saga ofrece el mejor balance simplicidad/robustez: su latencia en éxito (29.27 ms) es comparable a 2PC, y en escenarios de fallo la compensación añade solo 8 ms. Para sagas de más de 2 pasos, el número de compensaciones crecería linealmente con la longitud de la saga.

TCC mostró el mayor overhead (114 ms) debido a 4 operaciones adicionales por transacción. Sin embargo, su modelo de reservas ofrece una garantía más fuerte que Saga: los fondos están bloqueados desde TRY, eliminando condiciones de carrera en escenarios de alta contención.

Raft fue el único protocolo con 100% de éxito en todos los escenarios. La latencia de 95 ms refleja el costo de replicar cada comando a 3 nodos. Las limitaciones de la implementación —líder fijo, log en memoria, sin heartbeats— implican que estos resultados representan una cota inferior del rendimiento real. En producción, Raft añadiría 150–300 ms por elección de líder en caso de fallo del nodo líder.

Las amenazas a la validez incluyen: (1) todas las pruebas se ejecutaron en un solo host Docker sin latencia de red real, subestimando tiempos absolutos pero preservando diferencias relativas; (2) el tamaño de muestra (n=15) es adecuado para tendencias pero insuficiente para significancia estadística formal; (3) la simulación de fallos es determinista y no captura la estocasticidad de fallos reales.

## VI. CONCLUSIONES

Se implementaron y compararon experimentalmente cuatro protocolos de coordinación distribuida sobre PostgreSQL en un entorno Docker de 3 nodos (240 transacciones en total). Los hallazgos principales son:

- 2PC ofrece la menor latencia sin fallos (30 ms) pero su coordinador centralizado es un punto único de fallo y el timeout penaliza severamente los escenarios con pérdida de conectividad.

- Saga logra el mejor compromiso simplicidad/robustez: compensación correcta en 100% de los casos con solo 8 ms de overhead adicional.
- TCC proporciona garantías más fuertes mediante reservas explícitas, al costo de 3.8× más latencia. Es adecuado para escenarios con alta probabilidad de condiciones de carrera.
- Raft fue el único protocolo con 100% de tasa de éxito en todos los escenarios, demostrando empíricamente la tolerancia a fallos que el consenso mayoritario proporciona.

## VII. TRABAJO FUTURO

- Implementar la optimización Presumed Abort en 2PC para reducir mensajes en el caso de rollback.
- Replicar el coordinador 2PC con Paxos o Raft para eliminar el punto único de fallo.
- Evaluar los protocolos con latencias de red reales (>50 ms entre continentes) usando *tc-netem* en los contenedores Docker.
- Extender la implementación de Raft con elección dinámica de líder, heartbeats periódicos y persistencia del log en disco.
- Aumentar el tamaño de muestra a 100+ repeticiones para obtener significancia estadística en las diferencias de latencia.
- Integrar middleware de mensajería (RabbitMQ/Kafka) para desacoplar la comunicación entre nodos en Saga y TCC.

## REFERENCES

- [1] X/Open Company Ltd., "Distributed Transaction Processing: The XA Specification," X/Open Document XO/CAE/91/300, 1991.
- [2] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Reading, MA, USA: Addison-Wesley, 1987.
- [3] C. Mohan, B. Lindsay, and R. Obermarck, "Transaction Management in the R\* Distributed Database Management System," *ACM Transactions on Database Systems*, vol. 11, no. 4, pp. 378–396, Dec. 1986.
- [4] H. Garcia-Molina and K. Salem, "Sagas," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, San Francisco, CA, USA, 1987, pp. 249–259.
- [5] P. Helland, "Life beyond Distributed Transactions: an Apostate's Opinion," in *Proceedings of the Conference on Innovative Data Systems Research (CIDR)*, Asilomar, CA, USA, 2007.
- [6] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," in *Proceedings of the USENIX Annual Technical Conference (ATC)*, Philadelphia, PA, USA, 2014, pp. 305–319.
- [7] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*, 4th ed. New York, NY, USA: Springer, 2020.

## VIII. APÉNDICE

### A. Código Fuente

Repositorio del Proyecto

### B. Reproducibilidad

Todos los experimentos son reproducibles mediante:

- 1) `cd docker && docker compose up -d --build`
- 2) `cd experiments && sh run_experiments.sh`

3) `python analyze_results.py results graphs`

El código fuente completo está disponible en el repositorio del laboratorio.

a) *Requisitos previos:*

| Herramienta, Versión mínima, Verificación |       |                                     |
|-------------------------------------------|-------|-------------------------------------|
| Docker                                    | 24+   | <code>docker --version</code>       |
| Docker Compose                            | v2    | <code>docker compose version</code> |
| Python                                    | 3.11+ | <code>python --version</code>       |
| Bash                                      | 4+    | <code>bash --version</code>         |
| Typst (opcional)                          | 0.12+ | <code>typst --version</code>        |

Mapeo de puertos al host:

- `pg-arequipa` → `localhost:5432`
- `pg-lima` → `localhost:5433`
- `pg-cusco` → `localhost:5434`

b) *Ejecución de experimentos:*

Opción rápida: script con todos los escenarios

`cd articulo/scripts`

`chmod +x *.sh`

`./run_all.sh`

### C. Configuración de PostgreSQL

Parámetros relevantes de *postgresql.conf*: `wal_level = replica`, `max_wal_senders = 10`, `synchronous_commit = on`. Las cadenas de conexión usan el usuario *sdhm* sin contraseña y los hostnames de Docker: `pg-arequipa`, `pg-lima`, `pg-cusco` en puerto 5432.

### D. Datos de Prueba

Cuentas iniciales: Alvaro Quispe (Arequipa, S/ 50,000), Fabiana Pacheco (Lima, S/ 20,000), Mathias Barrios (Cusco, S/ 30,000). Las transferencias son por montos crecientes de S/ 500 a S/ 640 en incrementos de S/ 10.